

```
1 create database employeeAnalytics;
2 use employeeAnalytics;
3
4 -- drop table departments;
5 create table departments(
6     dept_id int primary key,
7     dept_name varchar(50) not null
8 );
9
10 -- drop table locations;
11 create table locations(
12     location_id int primary key,
13     location_name varchar(50) not null
14 );
15
16 create table job_roles(
17     role_id int primary key,
18     role_name varchar(50) not null,
19     min_salary decimal(10,2),
20     max_salary decimal(10,2)
21 );
22
23 -- drop table employees;
24
25 create table employees(
26     emp_id int primary key,
27     emp_name varchar(100) not null,
28     gender varchar(10),
29     dept_id int,
30     role_id int,
31     location_id int,
32     join_date date,
33     status varchar(20) default 'Active',
34     foreign key(dept_id) references departments(dept_id),
35     foreign key(role_id) references job_roles(role_id),
36     foreign key(location_id) references locations(location_id)
37 );
38
39 create table salaries(
40     salary_id int primary key,
41     emp_id int,
42     salary decimal(10,2),
43     effective_date date,
44     foreign key(emp_id) references employees(emp_id)
45 );
46
47 create table performance_reviews(
48     review_id int primary key,
49     emp_id int,
50     review_year int,
51     rating decimal(2,1) check (rating between 1 and 5),
52     foreign key(emp_id) references employees(emp_id)
53 );
```

```
54
55 truncate table employees;
56
57 BULK INSERT employeeAnalytics.dbo.performance_reviews
58 FROM 'C:\SQL-Data\Employeeanalytics - Performance_reviews extras.csv'
59 WITH (FIRSTROW = 2,
60       FIELDTERMINATOR = ',',
61       ROWTERMINATOR='\n' );
62
63 select * from performance_reviews order by rating desc;
64
65
66 --Performance analytics
67
68 --top 10 high performing employees
69
70 select top 10 employees.*, rating from employees
71 join performance_reviews on employees.emp_id =
    performance_reviews.emp_id
72 order by rating desc;
73
74 --average rating per department
75
76 select distinct employees.dept_id, avg(rating) over (partition by
    dept_id order by dept_id desc)
77 from employees
78 join performance_reviews on employees.emp_id =
    performance_reviews.emp_id;
79
80 --employees eligible for promotion(rating > 4 and 2+ years)
81
82 select employees.* from employees join performance_reviews on
83 employees.emp_id = performance_reviews.emp_id
84 where DATEDIFF(year,join_date,SYSDATETIME()) >= 2 and rating > 4;
85
86 --performance trend over years
87
88 select review_year, avg(rating) from performance_reviews group by
    review_year;
89
90
91 --Salary analytics
92
93 --salary distribution per department
94
95 select departments.dept_id, dept_name, salaryDistribution
96 from (select dept_id, sum(salary) as salaryDistribution from salaries
    join employees on salaries.emp_id = employees.emp_id
97 group by dept_id) as distribut join departments on departments.dept_id
    = distribut.dept_id;
98
99 --highest and lowest salary per role
100
```

```
101 select job_roles.role_id, max(salary) as highestSalary, min(salary) as lowestSalary
102 from job_roles join employees on employees.role_id = job_roles.role_id
103 join salaries on employees.emp_id = salaries.emp_id
104 group by job_roles.role_id;
105
106 --salary vs performance correlation
107
108 SELECT
109     (COUNT(*) * SUM(salary * rating) - SUM(salary) * SUM(rating)) /
110     (SQRT(COUNT(*) * SUM(POWER(salary, 2)) - POWER(SUM(salary), 2)) *
111     SQRT(COUNT(*) * SUM(POWER(rating, 2)) - POWER(SUM(rating), 2)))
112     AS correlation_coefficient
113 FROM employees
114 JOIN salaries ON employees.emp_id = salaries.emp_id
115 JOIN performance_reviews ON performance_reviews.emp_id = employees.emp_id;
116
117 --employees below department average salary
118
119 select employees.*, averageDeptSalary
120 from (select dept_id, avg(salary) as averageDeptSalary from salaries
121      join employees on salaries.emp_id = employees.emp_id
122      group by dept_id) as average join employees on average.dept_id = employees.dept_id
123 where salary < averageDeptSalary;
124
125
126 --Department growth
127
128 --hiring trend per year
129
130 select year(join_date), count(*) from employees group by year(join_date);
131
132 --headcount growth by department
133
134 select dept_id, year(join_date) as yearOfJoining, count(*) as empCount
135 from employees group by year(join_date), dept_id
136 order by dept_id, year(join_date);
137
138 --attrition rate by department
139
140 select departments.dept_id, dept_name, count(*)
141 from departments join employees on departments.dept_id = employees.dept_id
142 where employees.status = 'Resigned'
143 group by departments.dept_id, dept_name;
144
145
146 --Strategic insights
147
```

```
148 --employee overdue for promotion
149
150 select employees.*, rating from employees
151 join performance_reviews on employees.emp_id = performance_reviews.emp_id
152 where datediff(year, join_date, sysdatetime()) > 5
153 order by rating desc;
154
155 --high salary but low performance cases
156
157 select employees.*, salary, rating from employees
158 join performance_reviews on employees.emp_id = performance_reviews.emp_id
159 join salaries on salaries.emp_id = employees.emp_id
160 where rating < 4.0
161 order by salary desc, rating asc
162
163 --most stable department(lowest attrition)
164
165 select departments.dept_id, dept_name, count(*)
166 from departments left join employees on departments.dept_id = employees.dept_id
167 where employees.status = 'Resigned'
168 group by departments.dept_id, dept_name
169 order by count(*) ;
170
171 --location-wise salary comparison
172
173 select locations.location_id, location_name, sum(salary)
174 from employees
175 join salaries on employees.emp_id = salaries.emp_id
176 join locations on locations.location_id = employees.location_id
177 group by locations.location_id, location_name;
178
179 --gender diversity ratio (if added)
180
181 select gender, count(*) from employees group by gender;
182
183 --median salary calculation
184
185 WITH RankedSalaries AS (
186     SELECT
187         salary,
188         ROW_NUMBER() OVER (ORDER BY salary) as ranking,
189         COUNT(*) OVER () as total_count
190     FROM salaries
191 )
192 SELECT
193     ROUND(AVG(salary), 2) AS median_salary
194 FROM RankedSalaries
195 WHERE ranking IN (FLOOR((total_count + 1) / 2.0), ROUND((total_count + 1) / 2.0, 0));
196
```

197 --identify leadership pipeline candidates

198

199 select employees.* from employees join performance_reviews on
performance_reviews.emp_id = employees.emp_id



200 where rating > 4.5;